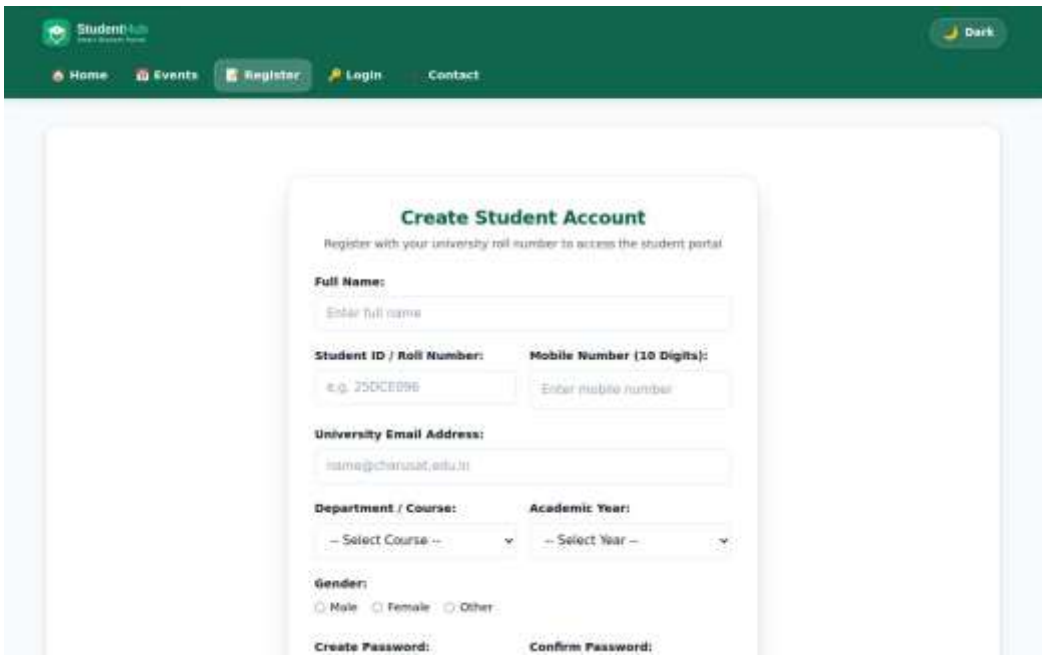
PRACTICAL-9

AIM: Implement secure user registration: validate input on the frontend and backend, check for duplicate e-mail and student ID with a prepared statement, store the password only as a password_hash() value, and show clear success and error messages.

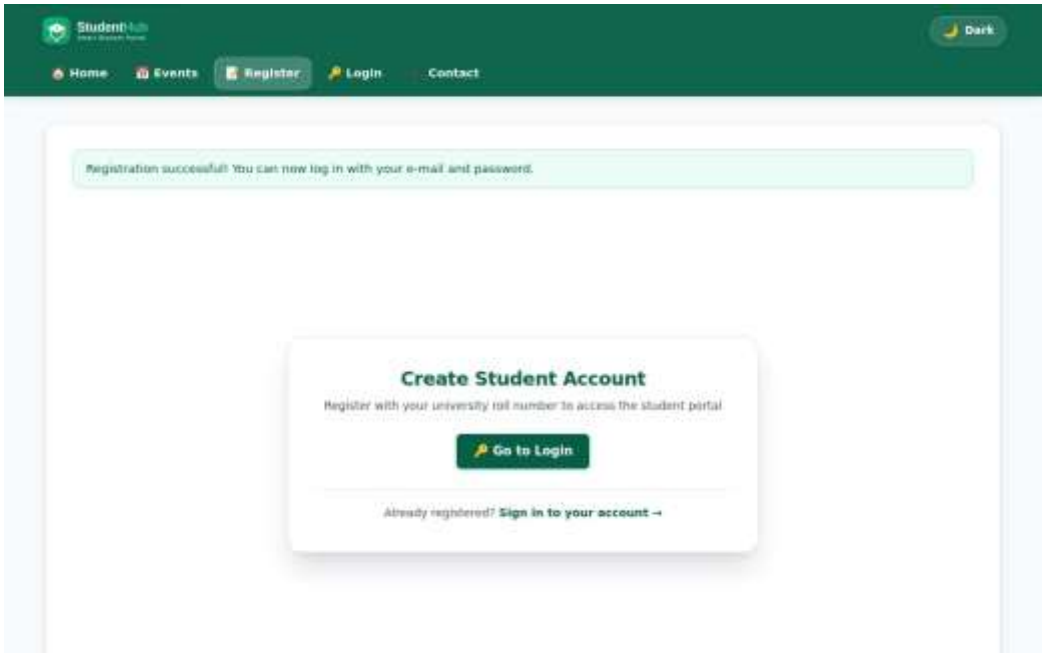
SOURCE CODE :

https://github.com/rachhadiyabrind-del/WDF_B/tree/main/Practical(1-4)(Studenthub)/Practical-9

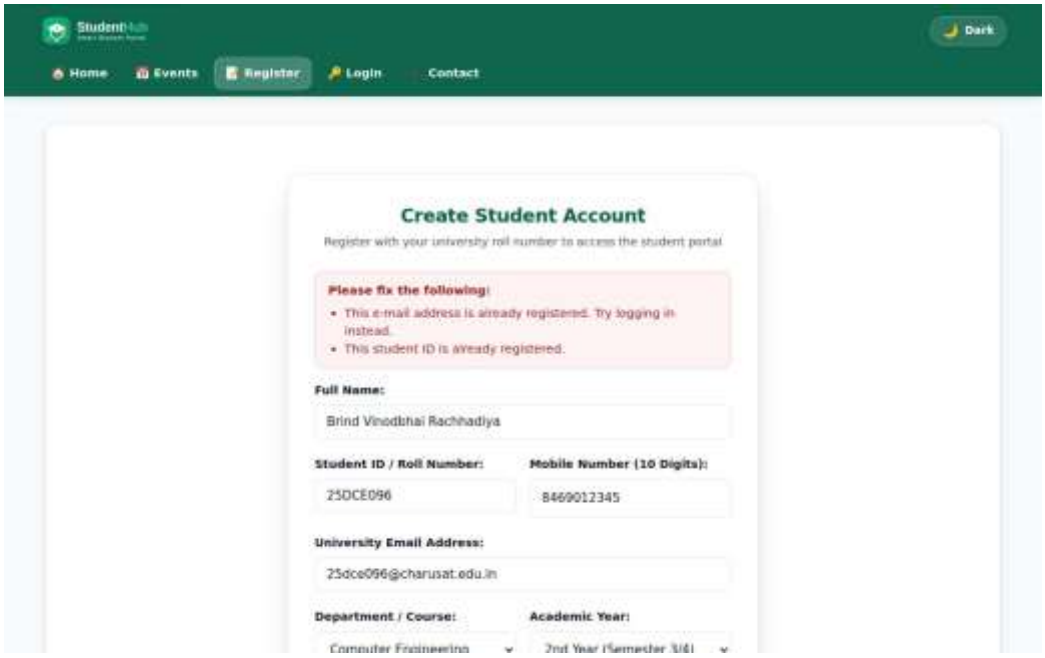
PREVIEW:



Registration form



Valid registration - success message



Same e-mail registered again - duplicate rejected



Invalid data sent to the server - full error list



mysql> SELECT ... FROM users (password is stored only as a bcrypt hash)

id	student_id	email	role	password_hash_first_40_chars	hash_length
5	25DCE096	25dce096@charusat.edu.in	student	$2y$10$bMwARozChqXAVf1Hy5VJnOvn2cln6gnwU	60

users table - password stored only as a bcrypt hash

TEST CASES :

#	Input / action	Expected result	Result
1	Register a new student with valid details	Success message; new row in the users table	Pass
2	Register with an e-mail address that already exists	"This e-mail address is already registered" error; no insert	Pass
3	Register with a student ID that already exists	"This student ID is already registered" error; no insert	Pass
4	Password without an uppercase letter or digit	Password pattern error from the server	Pass
5	Confirm password different from password	"Password and confirm password do not match" error	Pass
6	Leave the terms checkbox unticked	"You must accept the academic terms" error	Pass

7	Send invalid data to the server (browser checks bypassed)	Server returns the full list of errors; nothing stored	Pass
8	Submit the form without a valid CSRF token	Request rejected with a session-expired message	Pass
9	Inspect the password column in users	Value starts with $2y$ (bcrypt); plain text is not present	Pass

KEY QUESTIONS :

1. Is password_hash() used before storing passwords?

Yes. The password is hashed with PASSWORD_DEFAULT (currently bcrypt, with a random salt built in) before the INSERT. The plain-text password is never written to the database or the log, and only the hash is bound to the query.

```
$hash = password_hash($password, PASSWORD_DEFAULT);
```

2. Is duplicate e-mail or username checked?

Yes, in two layers. First, a prepared SELECT looks for an existing row with the same e-mail or student ID and shows a specific message. Second, the database has UNIQUE keys on email and student_id, so two requests submitted at the same time still cannot create duplicates. If MySQL reports error 1062 (duplicate key) during the insert, the code shows the same friendly message.

3. Are frontend and backend validations consistent?

The browser checks (required fields, minlength, pattern) give instant feedback, but they can be bypassed, so the server repeats the same rules: the name and student-ID patterns, the 10-digit mobile check, the whitelist for course and year, the password pattern and the password match. The server result is the final word, which is why the test with invalid data sent directly to the server still shows the full error list.